MEMORIA TÉCNICA: EPICVAULT

Autor: Darío Bonilla Perea

Proyecto: Gestión de Campañas de Rol

Tecnologías: Spring Boot 3, MySQL, Thymeleaf, JWT

Apartado 1: Documentación (15%)

1.1 Definición y Objetivo

EpicVault es una herramienta diseñada para directores de juego (DMs) que necesitan centralizar la información de sus partidas. El objetivo es permitir el registro de usuarios, la creación de campañas y el control de los personajes que participan en ellas, gestionando niveles e inventarios de forma persistente.

1.2 Requisitos Funcionales

- Registro de usuarios con credenciales únicas.
- Creación de campañas de rol con nombre único (RN-006).
- Asignación de personajes a campañas y usuarios específicos.
- Modificación de atributos de personajes (Subida de nivel).
- Visualización de datos mediante interfaz web y API JSON.

1.3 Modelo de Datos y UML

El sistema utiliza un mapeo ORM con tres entidades:

- **Usuario:** Representa a la persona física. (Relación N:M con Campaña).
- **Campaña:** Representa la sala de juego. (Relación 1:N con Personaje).
- **Personaje:** Representa la ficha de juego.

Apartado 3: Diseño de Servicios (20%)

3.1 Catálogo de Servicios Web

Nombre del Servicio	Ruta URL	Método	Entrada	Modo de Acceso	Formato Respuesta	Código	Descripción
Página Inicio	/	GET	Ninguna	Navegador	HTML	200	Menú principal.

Alta Usuario	/usuarios/guardar	POST	Body (Form)	Formulario	HTML	302	Crea usuario.
Alta Campaña	/campanas/guardar	POST	Body (Form)	Formulario	HTML	302	Crea campaña.
Lista JSON	/campanas/api/json	GET	Ninguna	API/Postman	JSON	200	Lista de campañas.
Saludo Cadena	/personajes/saludo	GET	Ninguna	Navegador	CADENA	200	Mensaje bienvenida.
Subir Nivel	/personajes/subir-nivel/{id}	POST	Param (id)	Postman	JSON	200	Suma +1 al nivel.

3.2 Gestión de Excepciones

Se ha implementado un ManejadorExcepciones (@ControllerAdvice) que captura cualquier error interno.

- **Navegación:** Ante un error, el sistema redirige a una vista personalizada error.html que muestra un mensaje intuitivo sin información técnica sensible.

Apartado 4: Operaciones de Acceso a Datos (25%)

La persistencia se realiza mediante **JpaRepository**. La base de datos se denomina BonillaYSpringBoot.

Apartado 6: Securización JWT (50% PSP)

Para este apartado se ha realizado una copia del proyecto implementando **Spring Security** y **jjwt**.

6.1 Secuencia de comprobación

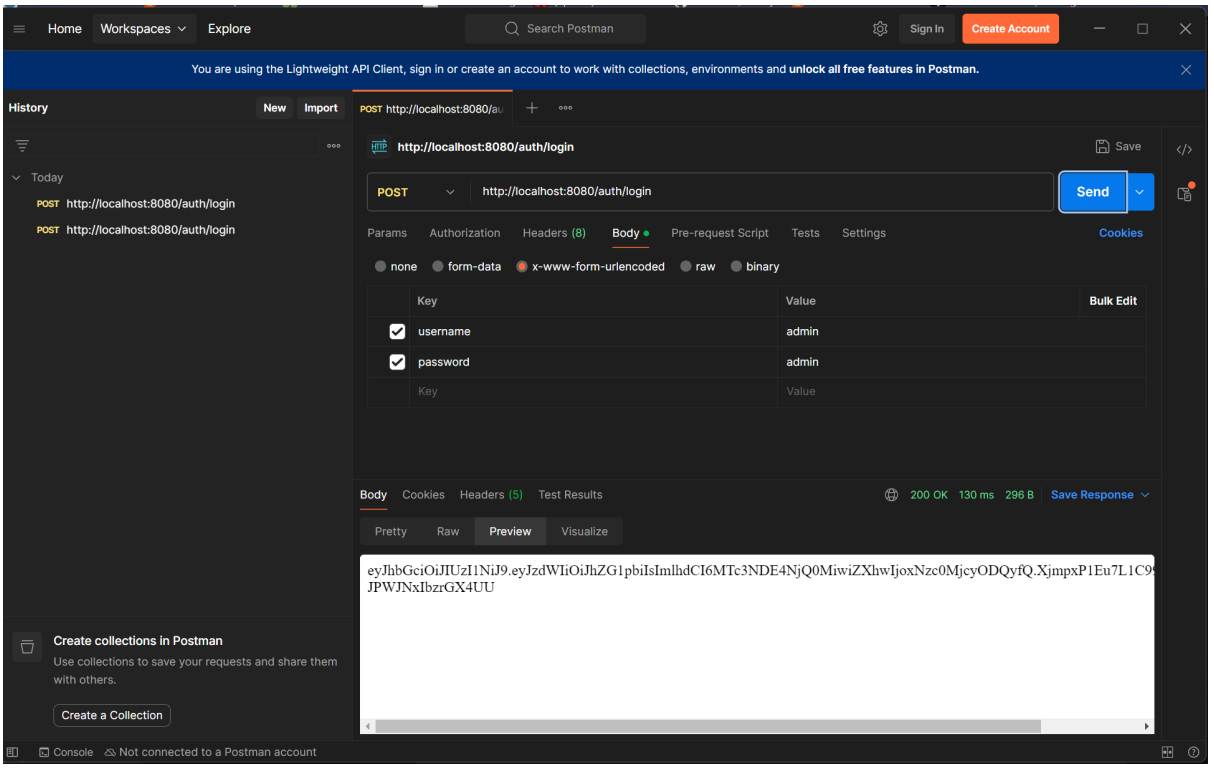
Escenario 1: Acceso No Autorizado (403)

Se intenta acceder a la lista de campañas sin proporcionar un token.

- **Ruta:** GET /campanas/api/json
- **Código esperado:** 403 Forbidden.

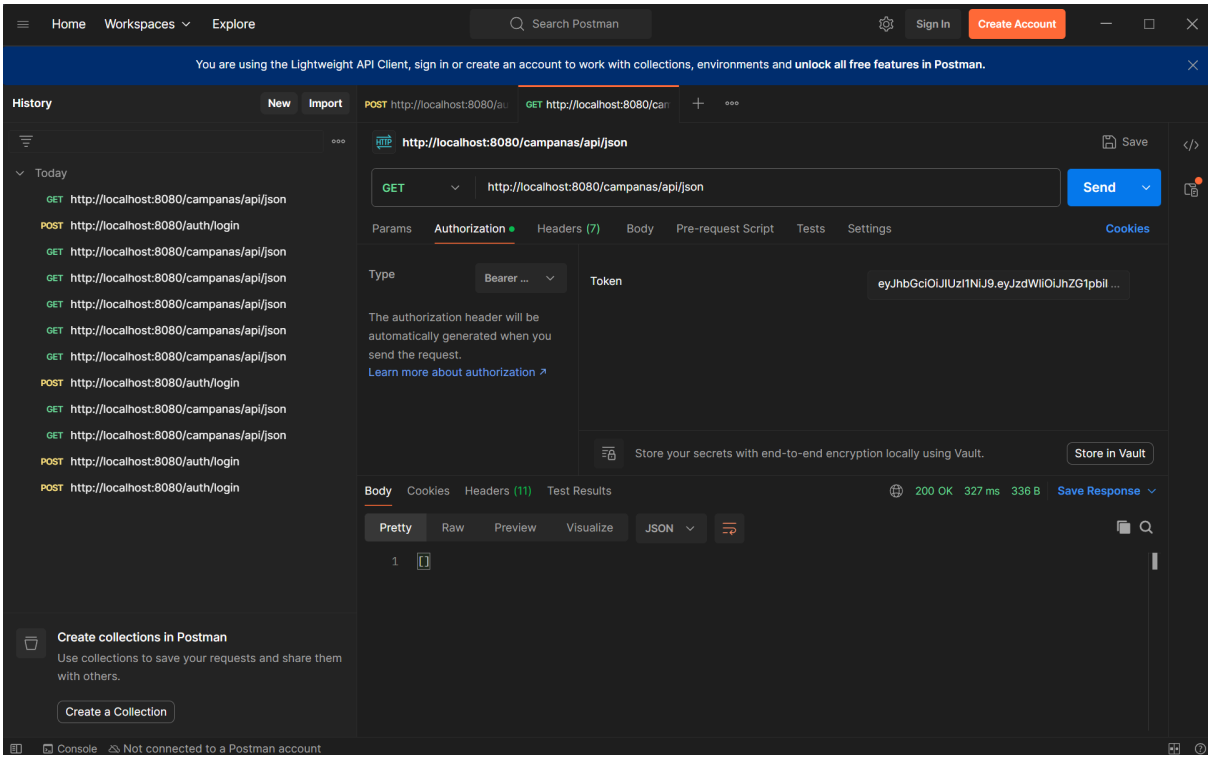
Escenario 2: Login e Identificación (200 OK)
Se envían credenciales válidas al servicio de autenticación.

- **Ruta:** POST /auth/login (Params: admin/admin)
- **Código esperado:** 200 OK.



Escenario 3: Acceso con Token Bearer (200 OK)
Se accede al servicio restringido enviando el token en el Header.

- **Header:** Authorization: Bearer <token>
- **Código esperado:** 200 OK.



Escenario 4: Credenciales Incorrectas (401)

- **Ruta:** POST /auth/login con datos erróneos.
- **Código esperado:** 401 Unauthorized.